ELEC 442 – Lab 1: Forward and Velocity Kinematics

Pre-lab, post-lab computations and concept questions

Exercise 1 – DH parameters and table

From Figure 2: $L_1 = 0.14$, $L_2 = 0.35$, $L_3 = 0.05$, $L_4 = 0.25$, $L_5 = 0.15$ m and $\beta = 8.13^\circ = \arctan(L_3/L_2)$. Two derived lengths are used:

$\lambda_2 = \sqrt{L_2^2 + L_3^2} = 0.3536$ m (shoulder to elbow), $\lambda_3 = L_4 + L_5 = 0.40$ m (elbow to gripper tip).

The joint angles ϕ_i are the angles read from the arm; the DH angles include offsets that make $\phi = 0$ the home pose. At home the physical upper arm rises L_2 and then jogs forward L_3, so the shoulder-to-elbow line (length λ_2) is tilted forward by β from vertical. The offset $\theta_2 = \phi_2 - 90^\circ + \beta$ aligns x_2 with that line, and the $-\beta$ in θ_3 cancels the tilt so that the forearm axis is horizontal at $\phi = 0$ (the forearm direction depends only on $\phi_2 + \phi_3$).

Link	θ_i	d_i (m)	a_i (m)	α_i
1	ϕ_1	$L_1 = 0.14$	0	-90°
2	$\phi_2 - 90^\circ + \beta$	0	$\lambda_2 = 0.3536$	0
3	$\phi_3 - \beta$	0	0	-90°
4	ϕ_4	$\lambda_3 = 0.40$	0	0

Check. At home ($\phi = 0$) this table gives ${}^0o_4 = [L_3 + L_4 + L_5,\ 0,\ L_1 + L_2]^T = [0.45,\ 0,\ 0.49]^T$ m, matching the handout. Both given test cases are reproduced to two decimals (Exercise 2).

Exercise 2 – Forward kinematics

Each link transform is

$$ {}^{i-1}T_i = \begin{pmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad {}^0T_4 = {}^0T_1\,{}^1T_2\,{}^2T_3\,{}^3T_4, $$

with location $= {}^0T_4[0{:}3, 3]$ and rotation $= {}^0T_4[0{:}3, 0{:}3]$.

```
import numpy as np
def dh(th, d, a, al): # i-1 T i from one DH row
```

```
    ct, st, ca, sa = np.cos(th), np.sin(th), np.cos(al), np.sin(al)
    return np.array([[ct, -st*ca, st*sa, a*ct],
                     [st, ct*ca, -ct*sa, a*st],
                     [0, sa, ca, d],
                     [0, 0, 0, 1]])

def forward_kin(phi): # phi: 4 joint angles [rad]
    L1, L2, L3, L4, L5 = 0.14, 0.35, 0.05, 0.25, 0.15
    beta = np.arctan2(L3, L2)
    lam2, lam3 = np.hypot(L2, L3), L4 + L5
    T = (dh(phi[0], L1, 0, -np.pi/2) @
         dh(phi[1]-np.pi/2+beta, 0, lam2, 0) @
         dh(phi[2]-beta, 0, 0, -np.pi/2) @
         dh(phi[3], lam3, 0, 0))
    return T[:3, 3], T[:3, :3] # location, rotation
```

Given test cases (my function reproduces the handout values):

Input (deg)	Location $[x, y, z]^T$ (m)	Rotation matrix
$[0, 0, 0, 0]$	$\begin{pmatrix} 0.45 \\ 0.00 \\ 0.49 \end{pmatrix}$	$\begin{pmatrix} 0.00 & 0.00 & 1.00 \\ 0.00 & -1.00 & 0.00 \\ 1.00 & 0.00 & 0.00 \end{pmatrix}$
$[30, 55, -60, 0]$	$\begin{pmatrix} 0.62 \\ 0.36 \\ 0.33 \end{pmatrix}$	$\begin{pmatrix} -0.08 & 0.50 & 0.86 \\ -0.04 & -0.87 & 0.50 \\ 1.00 & 0.00 & 0.09 \end{pmatrix}$

Recorded configurations:

Input (deg)	Location $[x, y, z]^T$ (m)	Rotation matrix
$[0, -11, -75, 0]$	$\begin{pmatrix} 0.01 \\ 0.00 \\ 0.89 \end{pmatrix}$	$\begin{pmatrix} -1.00 & 0.00 & 0.07 \\ 0.00 & -1.00 & 0.00 \\ 0.07 & 0.00 & 1.00 \end{pmatrix}$
$[-90, 73.5, -75, -50]$	$\begin{pmatrix} 0.00 \\ -0.75 \\ 0.20 \end{pmatrix}$	$\begin{pmatrix} 0.77 & -0.64 & 0.00 \\ 0.02 & 0.02 & -1.00 \\ 0.64 & 0.77 & 0.03 \end{pmatrix}$
$[-90, 73.5, -75, 0]$	$\begin{pmatrix} 0.00 \\ -0.75 \\ 0.20 \end{pmatrix}$	$\begin{pmatrix} 0.00 & -1.00 & 0.00 \\ 0.03 & 0.00 & -1.00 \\ 1.00 & 0.00 & 0.03 \end{pmatrix}$
$[90, -85, 75, 90]$	$\begin{pmatrix} 0.00 \\ 0.05 \\ 0.29 \end{pmatrix}$	$\begin{pmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 0.17 & 0.98 \\ 0.00 & -0.98 & 0.17 \end{pmatrix}$

Configurations 2 and 3 differ only in the wrist angle, so the location is identical and only the rotation changes (a rotation about the tool axis).

Exercise 3 – Velocity kinematics concept check

1. The joint motors control joint variables: angle (and angular velocity or torque) of the base, shoulder, elbow and wrist. No motor produces motion along x, y or z directly. The tip velocity is a configuration-dependent linear map of the joint rates, $v = J(q)\dot{q}$, so to get a desired Cartesian velocity the controller must solve for the joint rates, $\dot{q} = J^{-1}(q)\, v$ (with v extended by the orientation rate so that J is square, as in item 2). Since J depends on q, this is recomputed at every time step.

2. The task variables (p_x, p_y, p_z, φ) (here φ is the wrist orientation angle, not a joint angle) number four, equal to the four joints, so $\dot{x} = J_A(\theta)\dot{\theta}$ has a 4×4 square matrix. A square, non-singular matrix has an ordinary inverse, so we can solve for $\dot{\theta}$ exactly; at singular poses the inverse fails, and near them a damped or pseudo-inverse is needed. A full 6-DOF geometric Jacobian for a 4-joint arm is 6×4, has no inverse, and needs a pseudo-inverse (or SVD), which is costlier, only gives a least-squares answer, and cannot realise arbitrary 6-D motions with 4 joints anyway. The 4×4 inverse is cheap and of fixed cost, which suits real-time control; the trade-off is that only one orientation rate is controlled, so the full angular velocity vector is no longer available.

Exercise 6 – Effect of introduced errors (post-lab)

Positional error magnitude $\|p_{\text{err}} - p_{\text{true}}\|$ in cm, computed with my `forward_kin`. For the L_2 error I increased L_2 from 0.35 to 0.40 m and recomputed λ_2 and β from it; for L_4 I used 0.30 m.

Error type	Pose 1 (home)	Pose 2 (extended)	Pose 3 (retracted)
Shoulder joint $+10^\circ$	9.94	12.90	3.29
Elbow joint $+10^\circ$	6.97	6.97	6.97
L_2 length +5 cm	5.00	5.00	5.00
L_4 length +5 cm	5.00	5.00	5.00

Poses: $[0, 0, 0, 0]$, $[0, 55, -60, 0]$, $[0, -80, 70, 0]$ degrees.

Largest error and why it scales differently. The shoulder error is the largest in the home and extended poses (9.9 and 12.9 cm) but the smallest in the retracted pose (3.3 cm). A joint angle error $\Delta\phi$ moves the tip along an arc; the straight-line position error is the chord $2r \sin(\Delta\phi/2) \approx r\,\Delta\phi$, where r is the distance from that joint axis to the tip. The shoulder sees the whole arm, and r changes a lot with pose (about 0.57, 0.74, 0.19 m), so its error varies strongly: big when extended, small when retracted. The elbow only sees what is beyond it: the elbow-axis-to-tip distance is fixed at $\lambda_3 = 0.40$ m, so its error is always $2(0.40)\sin 5^\circ = 6.97$ cm regardless of pose. A link-length error is a rigid translation of everything beyond that link by ΔL along the link's own axis (for L_2, the vertical segment, whose direction depends on ϕ_2), so the magnitude is exactly 5.00 cm in every pose.

Worst-case pose. The shoulder error is maximal when the tip is as far as possible from the shoulder axis, i.e. with the shoulder, elbow and tip in a straight line. This happens for $\phi_3 = \beta - 90^\circ = -81.9^\circ$ (any ϕ_2), where $r = \lambda_2 + \lambda_3 = 0.754$ m and the error is $2(0.754)\sin 5^\circ = 13.14$ cm. A good test pose is [0, 85, −82, 0] degrees; the pose [0, 85, −90, 0] is 8° off straight and gives 13.10 cm (versus 12.90 cm for the extended pose above).

Exercise 7 – Post-lab questions

1. Drawbacks of cubic splines. A cubic spline gives continuous position and velocity (and acceleration for a C^2 spline such as SciPy's `CubicSpline`), but its third derivative, the jerk, is piecewise constant and jumps at every knot. Torque follows acceleration, so for a C^2 spline the torque is continuous but its rate of change jumps at each knot; if each segment is fitted separately with zero end velocities, the acceleration, and so the torque, jumps as well. Either way the sudden changes in motor torque or current excite vibration, cause wear in gears and joints, and stress the actuators. Other drawbacks: a spline can overshoot or wiggle between knots even though the knots are fine, it does not know about the joint velocity, acceleration or torque limits (or the workspace boundary), and forcing zero velocity at each knot makes the arm stop and start at every via-point.

2. Cumulative drift and feedback. Integrating $q \leftarrow q + J^{-1}(q)\, v\, dt$ is a first-order (Euler) approximation: the Jacobian is held constant over each step although it changes along the path, which gives a local truncation error of order dt^2 per step that accumulates to order dt over a run (floating-point round-off is much smaller, but also accumulates). Open loop, nothing ever compares the arm's actual position with the target, so each step's error is added to the previous ones and the arm drifts further each cycle. The command is the feed-forward spline velocity plus the term $K_p\big(\text{spline}(t) - \text{location}\big)$, which measures the actual tip position with forward kinematics and adds a velocity proportional to the tracking error. With $e = \text{spline} - \text{location}$ this gives $\dot{e} \approx -K_p e$, so the error decays exponentially (time constant $1/K_p$) instead of accumulating. This turns the open-loop integrator into a closed-loop tracker, and K_p must be tuned for stability: for the discrete update $e_{k+1} = (1 - K_p\, dt)\, e_k$ we need $0 < K_p\, dt < 2$ (and $K_p\, dt \leq 1$ to avoid oscillation), and in practice lower, because of the Jacobian approximation and actuator lag.

Not included

Exercise 4 (joint angles at home and zones 1–3) and **Exercise 5** (trajectory demo) need the physical or virtual QArm, so the values must be recorded in the lab. The individual reflection should be written by you personally.

Joint (Ex. 4)	home	1	2	3
base				
shoulder				
elbow				
wrist				